# Text-to-SQL con QLoRA en producción

Generación segura de consultas MySQL desde preguntas en español

**Objetivo**

Poner en producción el modelo desarrollado en el Proyecto 3 para que una persona externa pueda usarlo mediante una aplicación web. La adaptación conserva los datos, el modelo y las métricas, y añade validación de entradas, guardia SQL, telemetría privada y evidencia operativa.

**Wilson Alejandro Calderón Argueta** 22018
**Pablo Daniel Barillas Moreno** 22193

Deep Learning y Sistemas Inteligentes
2026

**Repositorio:**
https://github.com/DanielBarillasM/Proyecto-Final_Grupo-1_DLYSI_Sec-30

# Índice


**1. Resumen ejecutivo** **2**

**2. Problemática** **2**

**3. Planificación** **2**

**4. Supuestos** **2**

**5. Modelo planteado** **3**

**6. Validación de hipótesis** **3**

**7. Herramientas y arquitectura** **5**

7.1. Decisiones de despliegue . . . . . 5

**8. Evidencias del pase a producción** **5**

**9. Reproducibilidad y procedencia** **6**

**10. Limitaciones y recomendación** **6**

## 1. Resumen ejecutivo

El sistema recibe el DDL de un esquema y una pregunta en español, y devuelve una única consulta MySQL de solo lectura. El modelo base es Qwen2.5-Coder-1.5B-Instruct con un adaptador QLoRA `r=32`. En la validación heredada del Proyecto 3 alcanzó 97.4 % de exactitud ponderada por ejecución, frente a 3.8 % del modelo base.

El Proyecto Final no presenta un modelo nuevo ni repite el entrenamiento. El trabajo se concentra en la ruta de producción: interfaz pública, recuperación reproducible del adaptador, límites de entrada, rechazo de SQL destructivo, control de concurrencia y telemetría sin guardar contenido. El 5 de noviembre de 2026 la aplicación respondió HTTP 200 mediante un túnel temporal, produjo tres consultas correctas y cerró todos sus procesos después de reunir la evidencia.

## 2. Problemática

El usuario objetivo conoce el negocio y la estructura de sus datos, pero no necesariamente domina SQL. Hoy depende de un especialista para convertir cada pregunta operativa en una consulta. El modelo apoya la decisión de qué consulta de lectura utilizar para obtener la información solicitada.

**Alcance operativo**

La aplicación propone SQL para revisión. No solicita credenciales, no se conecta a una base de negocio y no ejecuta cambios. Una consulta segura todavía puede ser semánticamente incorrecta, por lo que el usuario debe revisarla antes de ejecutarla.

## 3. Planificación

| Fecha | Resultado | Puerta de calidad |
|---|---|---|
| 2–3 nov | Migración y trazabilidad | Código, datos y hashes verificados |
| 4 nov | Aplicación y controles | Flujo local, guardia SQL y telemetría |
| 5 nov | Despliegue | HTTP 200 público, tres casos y cierre verificado |
| 5 nov | Evidencias y documentación | README con siete secciones y capturas |
| 5 nov | Presentación | PDF de diez diapositivas y ensayo |
| 9 nov | Congelamiento | Release candidata y contingencia local |
| 10–15 nov | Ajustes | Correcciones sin alterar el experimento |

Pablo mantiene el runtime, la interfaz, el despliegue, las pruebas y la documentación. Wilson y su agente realizan una auditoría independiente cuyo único producto es `AUDITORIA_WILSON.md`. Ambos integrantes preparan la exposición oral. El repositorio heredado identifica dos integrantes; no se añadió una tercera persona ficticia.

## 4. Supuestos

- El usuario proporciona un DDL con al menos una sentencia `CREATE TABLE`.

- La pregunta está en español y se refiere a entidades presentes en el esquema.
- El dialecto objetivo es MySQL 8.4 y la salida debe ser una sola consulta de lectura.
- La demostración usa una RTX 4050 local; la primera solicitud incluye la carga del modelo.
- El servicio procesa una generación a la vez para proteger el runtime.
- Los datos sintéticos y dos esquemas inéditos no representan todos los dominios reales.

## 5. Modelo planteado

El proyecto parte del commit `df34c14c5afecddaa0d977832a56cec80c590244` del Proyecto 3. El modelo base y el adaptador se fijan mediante revisiones inmutables en `config.json`. El adaptador público es https://huggingface.co/Dani0604/proyecto3-text2sql-qlora.

La adaptación de producción introduce los siguientes cambios:

- descarga del adaptador desde Hugging Face cuando no existen pesos locales;
- interfaz Next.js publicada mediante Cloudflare y Gradio preparado para Spaces;
- validación de longitudes y presencia de `CREATE TABLE`;
- guardia que acepta únicamente `SELECT` o `WITH ... SELECT`;
- respuesta con latencia, variante e identificador de solicitud;
- telemetría que omite el esquema y la pregunta.

## 6. Validación de hipótesis

Antes del entrenamiento se definieron dos criterios: EX ponderada de al menos 0.70 y una mejora absoluta de al menos 0.15 frente al baseline.

| Variante | EX ponderada | Ejecutable | p95 |
|---|---|---|---|
| Qwen base | 3.8 % | 62.5 % | 6.66 s |
| QLoRA $r = 16$ | 97.4 % | 97.5 % | 15.72 s |
| **QLoRA** $r = 32$ | **97.4 %** | **97.5 %** | **13.20 s** |

La hipótesis de que `r=32` mejoraría la exactitud frente a `r=16` no se confirmó. Ambos obtuvieron 39 de 40 respuestas correctas. Se eligió `r=32` por menor latencia p95 después del empate. Frente al baseline, el adaptador sí superó ampliamente los criterios de éxito.

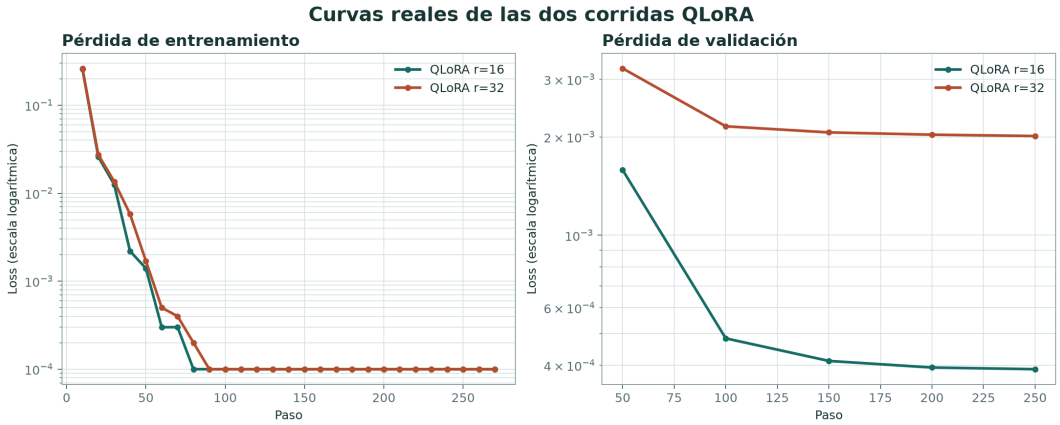



Figura 1: Pérdida de entrenamiento y validación de las dos configuraciones QLoRA. La selección final se basa en ejecución y latencia, no únicamente en esta curva.

## 7. Herramientas y arquitectura

1. Cloudflare reenvía la URL temporal a Next.js en `127.0.0.1:3000`.
2. Next.js usa un proxy restringido hacia FastAPI en `127.0.0.1:8000`.
3. La API valida estructura y tamaño antes de solicitar inferencia.
4. PyTorch, Transformers y PEFT cargan Qwen y QLoRA desde revisiones fijadas.
5. `sqlglot` normaliza la salida y rechaza escritura o sentencias múltiples.
6. Un log JSONL guarda fecha UTC, estado, latencia, tamaños y categoría de error.

FastAPI y Next.js forman la ruta pública temporal verificada. Gradio se conserva en el bundle mínimo generado por `scripts/build_space_bundle.py` para un futuro Space ZeroGPU. La cuenta de Hugging Face todavía no cumple la antigüedad mínima, por lo que no se simuló un despliegue permanente.

### 7.1. Decisiones de despliegue

| Decisión | Razón | Consecuencia |
|---|---|---|
| Next.js + Cloudflare | UI validada y túnel sin alterar el entorno CUDA | URL temporal por ejecución |
| Gradio para Spaces | Bundle compatible con ZeroGPU | Espera elegibilidad de la cuenta |
| Adapter remoto fijado | Evita duplicar un binario de 140.9 MB | Primera carga depende del Hub |
| Concurrencia uno | Protege memoria y estado del modelo | Menor capacidad simultánea |
| Logs sin contenido | Reduce exposición de esquemas | Diagnóstico mediante metadatos |

## 8. Evidencias del pase a producción

| Evidencia | Ubicación | Estado |
|---|---|---|
| Suite automatizada | `tests/` | 43 pruebas aprobadas |
| Smoke local RTX 4050 | `docs/evidencias/pruebas/` | 65.1 s, 1,472 MB |
| Evento anonimizado | `docs/evidencias/logs/` | Registrado |
| Aplicación pública | `docs/evidencias/despliegue.md` | HTTP 200 verificado |
| Capturas | `docs/evidencias/capturas/` | Dos imágenes revisadas |
| Resultados sintéticos | `docs/evidencias/pruebas/` | 3 de 3 correctos |
| Controles destructivos | `docs/evidencias/pruebas/` | 2 de 2 rechazados |
| Commit desplegado | `docs/evidencias/despliegue.md` | `0e304a20...66cf3a8` |

La URL temporal `https://bytes-aside-abraham-troy.trycloudflare.com` respondió HTTP 200. El caso fácil tardó 43.039 s con carga fría; los casos medio y difícil tardaron 3.014 s y 8.030 s. La API rechazó `DELETE` y una sentencia múltiple con HTTP 422. El túnel operó de 16:13 a 16:17, hora de Guatemala, y luego se verificó que los puertos de la demostración quedaran libres.

## 9. Reproducibilidad y procedencia

El dataset conserva 2,592 pares sobre 12 esquemas de entrenamiento y 40 preguntas sobre dos esquemas inéditos. Los hashes se mantienen en `resultados/dataset_manifest.json`. El adapter ganador conserva el SHA-256 `d3126183a092521cb1feffb9424b188552d27bb8fc4b51a8d13a1348d89016f1`.

La suite se ejecuta con:

```
python -m pytest -q
python -m ruff check production.py app.py text2sql tests
pnpm.cmd --dir web build
python scripts/build_space_bundle.py
python scripts/audit_final.py --final
```

## 10. Limitaciones y recomendación

El principal riesgo es el cambio de dominio. La validación cubre dos esquemas inéditos, pero no la diversidad de una organización real. El hardware bajo demanda también puede introducir cola. La recomendación es mantener el modelo para consultas supervisadas, conservar la restricción de solo lectura, exigir revisión humana y monitorear latencia, errores y dominios nuevos.

## Uso de inteligencia artificial

OpenAI Codex apoyó el análisis del enunciado, la migración, la implementación, las pruebas y la documentación. v0 se utilizó como punto de partida visual de la interfaz del Proyecto 3. Los integrantes revisan los cambios y son responsables de explicar el código y las decisiones. Ninguna API externa procesa las solicitudes del servicio; la inferencia utiliza el modelo y el adaptador abiertos descritos en este documento.